Alexia — Product Requirements Document

An autonomous DevSecOps guardian that writes, deploys, heals and protects your server — and never loses your data.

Hackathon	Nebius x NVIDIA Global AI Hackathon (Devpost)
Track	Coding and Agentic Engineering (+ Best Use of Tavily bonus)
Hard deadline	**Fri 30 Oct 2026, 10:00 am PT = 22:30 IST**
Our target submit time	Fri 30 Oct 2026, **18:00 IST** (4.5 h buffer)
Team	P1 — AI & Agent Lead · P2 — Platform & Reliability Lead · P3 — Security & Product Lead
Devpost Representative	P1 (submits on behalf of the team; see §4)
Version / date	v1.0 — 28 Oct 2026
Status	Approved for build

1. TL;DR

Small teams and solo developers run their projects on a single VPS with Docker. When something breaks — a bad deploy, a crashed container, a full disk, a leaked API key, an SQL-injection attack — nobody is watching, and the fix costs hours. Worse, a panicked fix can destroy data.

Alexia is a team of five cooperating AI agents running on **NVIDIA Nemotron models served by Nebius Token Factory**:

1. **Builder** — reviews commits, *writes code fixes, runs tests in a sandbox and opens pull requests*.
2. **Deployer** — ships code with health gates and automatic rollback.
3. **Healer** — detects outages and fixes them, then verifies the fix.
4. **Sentinel** — detects breaches (leaked secrets, brute force, SQL injection, data exfiltration), contains them and drafts the legally required breach report.
5. **Integrity** — snapshots data before any change, guards database migrations and restores data after corruption.

The headline loop the judges will see: **an attacker exploits an SQL injection → Sentinel detects and contains it → Integrity assesses exposed data → Builder writes the patch plus a regression test, runs it until it passes → Deployer ships it safely → a DPDP/CERT-In-ready breach report is drafted.** Every step is visible in a live dashboard with the model reasoning, the Tavily sources and the cost.

Nemotron **Nano** does fast triage, **Super** does diagnosis and code writing, **Ultra** does planning, investigation and review. **Tavily** grounds every diagnosis in live web knowledge (error messages, CVEs, docs).

The **Alexia Mobile companion** is an installable web app (PWA) with an optional Android build. It puts the server in your pocket:

- push notifications for incidents
- one-tap **Approve / Reject** for risky actions, protected by biometric or passkey confirmation
- a **Pause all autonomy** kill switch
- remote controls such as rollback, restart and block IP, all limited to the same safe action catalog

2. Problem and audience

2.1 The problem

- **Outages go unnoticed.** Indie developers, student teams and small startups rarely have on-call rotations or SRE staff. A crashed container can stay down for hours.
- **Fixes are slow and risky.** Diagnosing requires reading logs, Googling errors and remembering past fixes. Manual hot-fixes on production often make things worse, and data loss is the most expensive failure of all.
- **Security incidents are silent.** Leaked keys in git, brute-force attempts and injection attacks are usually discovered only after damage is done.
- **Breach reporting is legally time-boxed.** In India, CERT-In requires reporting of listed cyber incidents within **6 hours**; the DPDP Rules 2025 require notifying the Data Protection Board **without delay**, with a detailed report within **72 hours**. GDPR Art. 33 also

requires **72 hours**. Small teams do not know how to write these reports, let alone under pressure.

- **Existing tools stop at alerts.** Monitoring tools tell you *something* is wrong. They do not fix the code, prove the fix with a test, protect the data, or write the incident report.

2.2 Target users

Persona	Pain	What Alexia gives them
Indie developer / freelancer running 3–10 side projects on one VPS	Outages at night; no time for ops	Self-healing and safe deploys with zero setup beyond Docker Compose
Student / hackathon team shipping fast	Bad deploys, leaked keys in public repos	Pre-deploy code review, secret scanning, automatic rollback
Early-stage startup (2–15 people) handling user data	Breach risk and legal reporting duties with no security staff	Breach detection, containment, evidence and draft regulatory reports

2.3 Why now / why this stack

Open models such as Nemotron 3 now handle long contexts (262K–1M tokens) and tool use well enough to read whole log windows and diffs, and Token Factory makes them cheap enough to run on every incident: Nano costs about $0.06 / $0.24 per million input/output tokens.

---

3. Goals, non-goals and success metrics

3.1 Goals

- **G1:** Detect and automatically resolve common runtime failures on a Docker Compose host.
- **G2:** Close the loop from incident to code: write a patch, write a test that reproduces the bug, run it in a sandbox and open a PR.
- **G3:** Detect, contain and document security incidents, and draft compliant breach reports.
- **G4:** Guarantee data safety: no destructive action without a snapshot, and a tamper-evident audit trail.
- **G5:** Show the reasoning: every decision visible with its model, sources, cost and confidence.

3.2 Non-goals (out of scope for the hackathon)

- Kubernetes, multi-host clusters, or cloud-provider-specific integrations.
- Offensive security tooling or scanning of any system we do not own.
- Replacing a lawyer: breach reports are **drafts for human review**.
- Fully autonomous merging to production branches. Merges and secret rotation always need human approval.

3.3 Success metrics (measured in the demo environment)

Metric	Target
Mean time to detect (rule-based detectors)	**< 30 s**
Mean time to resolve, Tier-0 incidents (restart, rollback, cleanup)	**< 2 min**
Builder fix PRs where the generated test fails before the patch and passes after	**≥ 80 %** of code-bug scenarios
Demo scenarios passing 3 consecutive runs	**6 of 6** must-have/should-have scenarios
Forbidden actions executed	**0** (enforced in code, not by prompt)
Average model cost per incident	**< $0.05**, shown live on the dashboard
Audit-log hash chain verification	**100 %** valid after every demo run

---

4. Hackathon rules compliance matrix

Source: the official rules at nebiusglobalaihackathon.devpost.com/rules, read on 28 Oct 2026. Each rule has an owner and evidence that it was met.

#	Rule (official)	How Alexia complies	Owner	Evidence
R1	Must run on Nebius Token Factory or Nebius AI Cloud: "a runtime call to the Token Factory inference API"	Every agent calls Token Factory (`https://api.tokenfactory.nebius.com/v1/`) through one client in `alexia/core/llm.py`	P1	Code, dashboard model panel, video
R2	Use at least one NVIDIA open source model	Nemotron 3 Nano, Super and Ultra, routed by task	P1	`config/models.yaml`, README
R3	Fit a track; Stage One pass/fail: "a genuine attempt at the track's stated goal, not a superficial rebrand"	Coding track = "agents that write, run, and test code". The **Builder agent writing a patch, running tests and iterating is the centrepiece** of the demo, not a side feature	P1	Video 0:40–1:40, README
R4	Working demo URL, hosted app or test build	Public dashboard at `https://alexia-demo.<domain>` with "Run scenario" buttons against a sandbox target app	P2	URL in submission form
R5	Project must be **available free and without restriction until the Judging Period ends (15 Dec 2026, 12:00 pm PT)**	Demo VPS stays up until 16 Dec; uptime monitor; daily token budget cap; auto-reset of the target app	P2	Uptime monitor screenshot
R6	If the site is private, include login credentials in testing instructions	Dashboard demo is public (read + run scenarios). The mobile companion needs a login, so a **demo account (viewer + approver, sandbox only)** goes in the Devpost testing instructions (FR-M12)	P3	Devpost testing section
R7	Public repo on GitHub/GitLab/Bitbucket with **all source, assets and instructions**	Public GitHub monorepo, including target app, chaos scripts, dashboard and Compose files	P1	Repo URL
R8	Open source license **detectable and visible at the top of the repo (About section)**	`LICENSE` = MIT at repo root, added in the very first commit; check that GitHub shows "MIT license" in About	P1	Screenshot of repo About
R9	README with setup instructions and how to run	README sections: Quickstart, Configuration, Running the demo, Architecture, How we use Nemotron and Token Factory, Nebius feedback	P1 + P2	README
R10	Highlight how NVIDIA models and Token Factory were used, plus other Nebius tools	README section "How Alexia uses Nemotron on Token Factory" with the routing table, token and cost stats, Serverless Job usage (if built)	P1	README
R11	Demo video **< 3 min**, public on **YouTube**, shows the project working; **no third-party trademarks or copyrighted music without permission**	Target length 2:50. No music, or royalty-free music from the YouTube Audio Library. Only our own UI and terminal on screen	P3	YouTube link
R12	Identify the track	"Coding and Agentic Engineering"	P1	Submission form
R13	Provide feedback on Token Factory, AI Cloud and NVIDIA tools (also eligible for "Most Valuable Feedback", $100 × 10)	Running feedback log kept by all three (`docs/feedback.md`), edited into a specific, actionable section	All, edited by P1	Submission form + README
R14	New or significantly updated during 26 Aug – 30 Oct 2026	Alexia is **new**, with its first commit on 28 Oct 2026. Any existing personal project used as a monitored target is declared as a target only, not as part of the submission	P1	Git history
R15	Original work, solely owned, no IP violations; open source allowed if licenses are respected and we build on top	Only permissive dependencies (MIT, Apache-2.0, BSD). Use `httpx` for the GitHub REST API instead of an LGPL client. Dependency licenses listed in `THIRD_PARTY.md`	P2	`THIRD_PARTY.md`
R16	Third-party APIs (Tavily, GitHub) must be used per their terms	Free or credited Tavily plan used within its terms; GitHub fine-grained token scoped to the demo repo only	P1	`.env.example`

#	Rule (official)	How Alexia complies	Owner	Evidence
R17	No viruses or malicious code in the submission	Attack simulations are harmless scripts that **only** target our own sandbox containers (hard-coded allowlist). No exploit payloads against third parties	P3	`chaos/` README, code review
R18	All materials in English	Video narration, README and description in English	All	—
R19	Team: appoint one **Representative** who submits; prizes are paid to the Representative, who splits them	P1 is Representative. P2 and P3 accept Devpost team invites. **Agree the prize split in writing now** (default: equal thirds)	P1	Team chat message
R20	Submission cannot be edited after the deadline	Tag the submitted commit `v1.0-submission`; never force-push `main`; later work goes on a `post-hackathon` branch	P1	Git tag
R21	Eligibility: age of majority; not a resident of a sanctioned or excluded region; not employed by the sponsor or a judge's company	Each member confirms this privately before submitting	All	—

4.1 Prize strategy (from §8 of the rules)

- A project can win **one Overall award OR one Track award, plus one Bonus award**.
- We compete for **Overall / Coding Track** and the **Best Use of Tavily bonus ($3,000)**. That bonus requires "a functional, runtime call to the Tavily API": Tavily is used on every diagnosis.
- **Most Valuable Feedback ($100 + swag, 10 winners):** a complete, specific feedback section.
- **City Winner awards:** only for attendees of the listed Builders & Brews cities. No Indian city is listed, so we are **not eligible** unless a member attended one.
- **Credits:** join the **Nebius Builder Program** for Token Factory and Tavily credits.

---

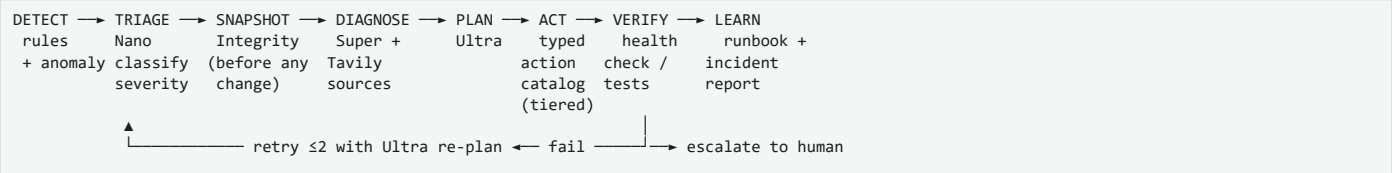
5. Product overview

5.1 The five agents

Agent	Watches	Does	Example
Builder (developer agent)	Git pushes, PRs, incidents traced to code	Reviews diffs before deploy; writes a patch **and** a regression test; runs the test suite in a sandbox; iterates up to 3 times; opens a PR with an explanation	SQL injection found → parameterised query + test that sends the payload → tests green → PR #12
Deployer	Git webhook on `main`	Builds the image, pre-deploy checks (Builder review, Sentinel secret scan, Integrity migration guard), deploys, health-gates for 3 min, rolls back on failure	A deploy causes 500s → rollback in 40 s → Builder asked to fix
Healer	Docker events, health endpoints, resource metrics, logs	Triage → diagnose → act from the action catalog → verify → write a runbook entry	Container out of memory → restart with a higher memory limit (needs approval)
Sentinel (breach agent)	Git diffs, app access and auth logs, DB query stats, processes and ports	Detects → contains → preserves evidence → investigates a timeline → drafts breach notices	400 failed logins from one IP → IP blocked for 1 h → report
Integrity	DB, volumes, migrations	Snapshot before any action; integrity checks; migration dry run on a DB copy; data restore; data-exposure assessment	Migration drops a column with data → blocked, approval required

The **Orchestrator** (Nemotron Ultra) coordinates agents when one incident spans several of them, for example a breach that needs a code fix and a deploy.

5.2 Core loop (every incident)



5.3 User stories

ID	As a…	I want…	So that…	Priority
US1	Developer	a crashed service restarted and verified automatically	my app is back up without me	Must
US2	Developer	bad deploys rolled back automatically	users never see a broken release for long	Must
US3	Developer	Alexia to write a fix with a test and open a PR	I review a proven fix, not a guess	Must
US4	Developer	leaked secrets detected in commits before deploy	my keys don't end up public	Must
US5	Startup founder	attacks detected, contained, and explained as a timeline	I know what happened and what data was exposed	Must
US6	Startup founder	a draft DPDP / CERT-In / GDPR breach report	I can meet legal deadlines	Must
US7	Developer	every risky action to need my one-click approval	the agent can't make things worse	Must
US8	Developer	a snapshot before any change and one-click restore	I never lose data	Must
US9	Developer	to see why the agent did something, with sources and cost	I can trust and audit it	Must
US10	Developer	repeated incidents fixed instantly from memory	it gets faster and cheaper over time	Should
US11	Developer	dangerous migrations blocked before they run	I don't lose columns or rows	Should
US12	Developer	a push notification on my phone when something happens	I know immediately, even away from my laptop	Should
US13	Developer	to approve or reject risky actions from my phone with a fingerprint or face check	the fix doesn't wait until I'm at a computer, and a stolen phone can't approve	Should
US14	Developer	a one-tap "Pause all autonomy" kill switch on my phone	I can stop the agent instantly if I don't trust what it's doing	Should
US15	Developer	to trigger safe remote actions (rollback, restart, block IP) from my phone	I can respond from anywhere without SSH	Should
US16	Developer	Alexia as an app on my home screen (Android/iOS), not just a website	it feels like a real product and notifications work	Should (PWA) / Could (native APK)

6. Functional requirements

Priority key: **M** = Must (needed for the demo), **S** = Should, **C** = Could (stretch goal).

6.1 Core platform (P1 owns schemas, P2 owns runtime)

ID	Requirement	Pri
FR-C1	Event bus (in-process asyncio queue + SQLite persistence) carrying `Event` objects per the schema in §8.4	M
FR-C2	`Incident` lifecycle: `open → triaged → acting → verifying → resolved / escalated`, every transition persisted	M
FR-C3	Single LLM client for Token Factory with model routing, retries, timeouts, token and cost accounting per call	M
FR-C4	**Redaction layer:** before any text is sent to a model, mask secrets (API-key patterns, JWTs, private keys, passwords in URLs), emails, phone numbers, 12-digit ID numbers and card numbers (Luhn check)	M
FR-C5	Structured outputs: every model response validated against a Pydantic schema; on failure, retry once then escalate the model tier	M
FR-C6	**Action catalog**: the model can only pick typed actions (§9.1); no raw shell commands are ever executed from model output	M
FR-C7	**Tiered approvals**: Tier 0 auto, Tier 1 needs approval (dashboard button; optional Telegram), Tier 2 forbidden (rejected in code)	M
FR-C8	**Tamper-evident audit log**: append-only table; each row stores `sha256(prev_hash + row_json)`; dashboard shows "chain verified OK"	M
FR-C9	**Runbook memory**: on resolution, store (incident signature → working fix). If a later incident matches, replay the fix with Nano confirmation and skip Ultra	S
FR-C10	Cost guard: daily token budget; when exceeded, switch to "replay mode" (cached runs) so the public demo never breaks	M

6.2 Builder agent (P1)

ID	Requirement	Pri
FR-B1	Pre-deploy review: given a diff, Super flags likely bugs, security issues and missing tests; Ultra decides block or allow for high-severity findings	M
FR-B2	Fix generation: given an incident with a code root cause, Super writes (a) a failing regression test and (b) a patch	M
FR-B3	**Run loop:** apply the patch in an isolated sandbox container (no network, CPU/RAM limits, 120 s timeout); run `pytest`; feed failures back; up to 3 iterations	M
FR-B4	Proof of fix: record that the test **fails before** and **passes after** the patch; attach both outputs to the PR	M
FR-B5	Open a PR on the target repo via the GitHub REST API: title, root cause, patch, test, Tavily sources, model trace link	M
FR-B6	Ultra reviews its own patch before the PR is opened (security and side-effect check)	S
FR-B7	Run the sandbox test suite as a **Nebius Serverless Job** instead of a local container (showcases Nebius AI Cloud)	C

6.3 Deployer agent (P2)

ID	Requirement	Pri
FR-D1	GitHub webhook (HMAC-verified) on push to `main` triggers a deploy	M
FR-D2	Pre-deploy gates: Builder review (FR-B1), Sentinel secret scan (FR-S1), Integrity migration guard (FR-I3)	M
FR-D3	Build a tagged image, keep the last 3 good images, deploy via Docker Compose	M
FR-D4	Health gate: for 3 min, check `/health`, 5xx rate and container restarts; on failure, automatic rollback (Tier 0)	M
FR-D5	Merging a Builder PR redeploys through the same gated pipeline	M

6.4 Healer agent (P2 actions, P1 reasoning)

ID	Requirement	Pri
FR-H1	Collectors: Docker events (die, oom, health_status), HTTP health polling every 10 s, disk/memory/CPU every 15 s, log tail with error-spike detection	M
FR-H2	Triage with Nano: classify into {crash, oom, disk_full, config_error, dependency, code_bug, unknown} plus severity and confidence	M
FR-H3	Diagnose with Super + Tavily: logs (redacted), recent diff, config; output root cause, evidence and sources	M
FR-H4	Plan with Ultra: pick action(s) from the catalog with expected outcome and rollback	M
FR-H5	Verify: re-check health; if not healthy within 60 s, retry (≤ 2) then escalate	M
FR-H6	Code-bug root causes are handed to Builder (FR-B2)	M

6.5 Sentinel agent (P3)

ID	Requirement	Pri
FR-S1	Secret scanning on every diff: regex rules + entropy; Nano confirms likely true positives	M
FR-S2	Brute-force detection: > N failed logins per IP per minute on the target app's auth log	S
FR-S3	Injection detection: request-log patterns (`' OR 1=1`, `UNION SELECT`, etc.) plus a DB-side anomaly (rows read per request > baseline × 10)	M
FR-S4	Exfiltration anomaly: unusual volume of personal-data rows returned or large egress from the app container	S
FR-S5	Containment (Tier 0): temporary IP block (1 h) at the reverse proxy. Tier 1: isolate the container from the network, rotate a secret, revoke tokens	M
FR-S6	Evidence preservation **before** containment changes state: copy relevant logs, hash them, store in an evidence folder, record in the audit log	M
FR-S7	Investigation with Ultra: timeline (first seen, entry point, actions, data touched), affected records count (from Integrity), confidence	M
FR-S8	Breach report drafts: **(a)** CERT-In incident report (6-hour clock), **(b)** DPDP Rules 2025 initial intimation and 72-hour detailed report to the Data Protection Board plus notice to affected Data Principals, **(c)** GDPR Art. 33 notice. All are marked "DRAFT — requires human/legal review"	M
FR-S9	Dependency CVE check: parse `requirements.txt`, query Tavily for known CVEs of pinned versions, open a PR to upgrade (via Builder)	C

6.6 Integrity agent (P2)

ID	Requirement	Pri
FR-I1	Snapshot before any Tier 0 or Tier 1 action that can touch data: `pg_dump` (target DB) + volume tarball, with a sha256 recorded in the audit log	M
FR-I2	Scheduled integrity checks: row counts, foreign-key checks, checksums of critical files, migration status	M
FR-I3	Migration guard: run the migration on a throwaway DB copy; diff the schema and row counts; Ultra reviews for destructive operations; block and require approval if data loss is detected	S
FR-I4	One-click restore to any snapshot (Tier 1) with a post-restore integrity check	M
FR-I5	Data-exposure assessment for Sentinel: which tables and columns, and how many rows, were readable by the attacker's queries	M

6.7 Dashboard (P3)

ID	Requirement	Pri
FR-U1	Live incident timeline (SSE), colour-coded by agent and severity	M
FR-U2	Incident detail: event, triage, diagnosis, Tavily sources, plan, actions, verification, cost, and a "reasoning trace" showing each model call (model, tokens, latency)	M
FR-U3	Approval panel: pending Tier 1 actions with Approve / Reject and the snapshot ID shown	M
FR-U4	Services panel: health of target services, last deploy, current image	M
FR-U5	Model and cost panel: calls and spend per model (Nano / Super / Ultra), total Tavily calls	M
FR-U6	Scenario launcher: "Run scenario" buttons (rate-limited; one at a time) plus "Reset environment"	M
FR-U7	Breach report viewer with export to Markdown/PDF	S
FR-U8	Audit log view with hash-chain verification status	M
FR-U9	Responsive layout; usable at 1280 px wide and on mobile for approvals	S

6.8 Mobile companion — website + app (P3 UI, P2 backend, P1 notification content)

Approach: one codebase. The React dashboard becomes an installable **Progressive Web App**: website and phone app in one, installable from the browser on Android and iOS, no app store needed. That also means judges can try it from a link. As a stretch goal, the same build is wrapped with **Capacitor** into a native Android APK, attached to a GitHub Release.

Screens (mobile-first)

Screen	Content
Home	System status (green/amber/red), open incidents, pending approvals badge, autonomy state (Active / Paused)
Incident	Plain-English one-line summary (Nano), timeline, root cause, Tavily sources, actions taken, cost
Approvals	Each pending Tier-1 action: what, why, risk, snapshot ID, expiry countdown; **Approve** / **Reject** with biometric or passkey confirmation
Controls	Pause / Resume all autonomy; rollback a service; restart a service; block or unblock an IP; take a snapshot; run an integrity check. Each control is a catalog action
Notifications	Feed of all alerts; per-severity mute settings; quiet hours
Devices & Security	Registered devices, last seen, revoke a device; recent remote actions from the audit log

Requirements

ID	Requirement	Pri
FR-M1	PWA: web app manifest, service worker, offline shell, installable on Android Chrome and iOS Safari (Add to Home Screen)	S
FR-M2	**Web Push notifications** (VAPID) for: new high or critical incident, approval needed, incident resolved, autonomy paused or resumed, deploy rolled back	S
FR-M3	Notification text: one sentence written by Nano from the redacted incident, **never containing secrets or personal data** (redaction runs first)	S
FR-M4	Actionable notifications: on Android, "Review" and "Reject" buttons in the notification. **Approve always opens the app** for step-up authentication. iOS opens the Approvals screen (iOS web push has no action buttons)	S
FR-M5	**Remote approvals** for Tier-1 actions, needing step-up authentication (WebAuthn passkey / device biometric, fallback TOTP) within the last 5 minutes	S
FR-M6	**Kill switch:** "Pause all autonomy" stops Tier-0 auto-actions as well (detection keeps running; actions queue for approval). Pausing needs only a logged-in session, because it is the safe direction. **Resuming needs step-up**	S
FR-M7	Remote controls go through the **same action catalog and tier checks** as the agents; the phone can never run arbitrary commands	S
FR-M8	Approval tokens are single-use, bound to `action_id` + device, expire after 5 minutes, and are signed server-side (HMAC) to prevent replay	S
FR-M9	Device registration and revocation; every remote action is written to the audit hash chain with device ID, user and IP	S
FR-M10	Fallback channel: Telegram bot for notifications only (no approvals over Telegram) when Web Push isn't available	C
FR-M11	Native Android build via Capacitor (same web code) with Firebase Cloud Messaging push; APK attached to a GitHub Release	C
FR-M12	**Demo mode for judges:** a shared demo account (viewer + approver) limited to the sandbox target, with credentials in the Devpost testing instructions; demo approvals use TOTP or skip step-up with a visible "demo" banner	S

Notification matrix

Event	Push?	Severity	Tap opens
Critical security incident (S6, S4)	Yes, always, ignores quiet hours	Critical	Incident
Approval needed	Yes	High	Approvals
Auto-resolved incident (S2, S3)	Optional (default: digest)	Low	Incident
Deploy rolled back	Yes	High	Incident
Autonomy paused or resumed	Yes, to all devices	Info	Home
Daily budget at 80 %	Yes	Medium	Settings

---

7. Model and tool usage

7.1 Nemotron routing on Token Factory

Base URL: `https://api.tokenfactory.nebius.com/v1/` (OpenAI-compatible). **Model IDs must be verified on day 1** with `GET /v1/models` (see §13). Keep them in `config/models.yaml`, never hard-coded.

Tier	Model (ID to verify)	Context	Price / 1M tokens (in/out)*	Used for
Nano	`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B` (alternative: `nvidia/Nemotron-3_5-Lightning`)	262K (Lightning: 1M)	$0.06 / $0.24	Event triage, log summarisation, secret-scan confirmation, runbook matching
Super	`nvidia/nemotron-3-super-120b-a12b` (catalog also lists `nvidia/Nemotron-3.5-Super-VL`)	256K	$0.30 / $0.90	Diagnosis, writing patches and tests, structured extraction, report drafting
Ultra	`nvidia/Nemotron-3-Ultra-550b-a55b`	1M	$1.00 / $3.00	Fix planning, breach investigation and timeline, patch review, migration risk review, multi-agent orchestration

*Prices from the Nebius Nemotron catalog page and third-party listings as of Oct 2026; confirm in the console.

Escalation rules

- Nano confidence < 0.7 → re-run with Super.
- Super's fix fails verification twice → Ultra re-plans.
- Ultra fails → escalate to a human with a full summary.
- Known runbook match → Nano confirms and replays (no Ultra call).

Cost estimate: a typical Tier-0 incident is about 8K Nano + 15K Super + 10K Ultra tokens, roughly **$0.04**. A full breach scenario is roughly $0.15–0.30. Budget for the hackathon plus judging: under $50, covered by Builder Program credits.

7.2 Tavily (Best Use of Tavily bonus)

- **Where:** Healer and Builder diagnosis (search the exact error message + framework version), Sentinel (attack technique and remediation guidance, CVE lookups), Integrity (DB engine error codes).
- **How:** `tavily-python` → `search(query, search_depth="basic", max_results=5)`; the top results' content goes into the Super prompt; URLs are shown as **Sources** on every incident.
- **Budget:** basic search = 1 credit; free tier = 1,000 credits/month plus Builder Program credits. Cache results by query hash for 24 h.
- **Showcase:** the video shows a Tavily source being the reason the fix worked.

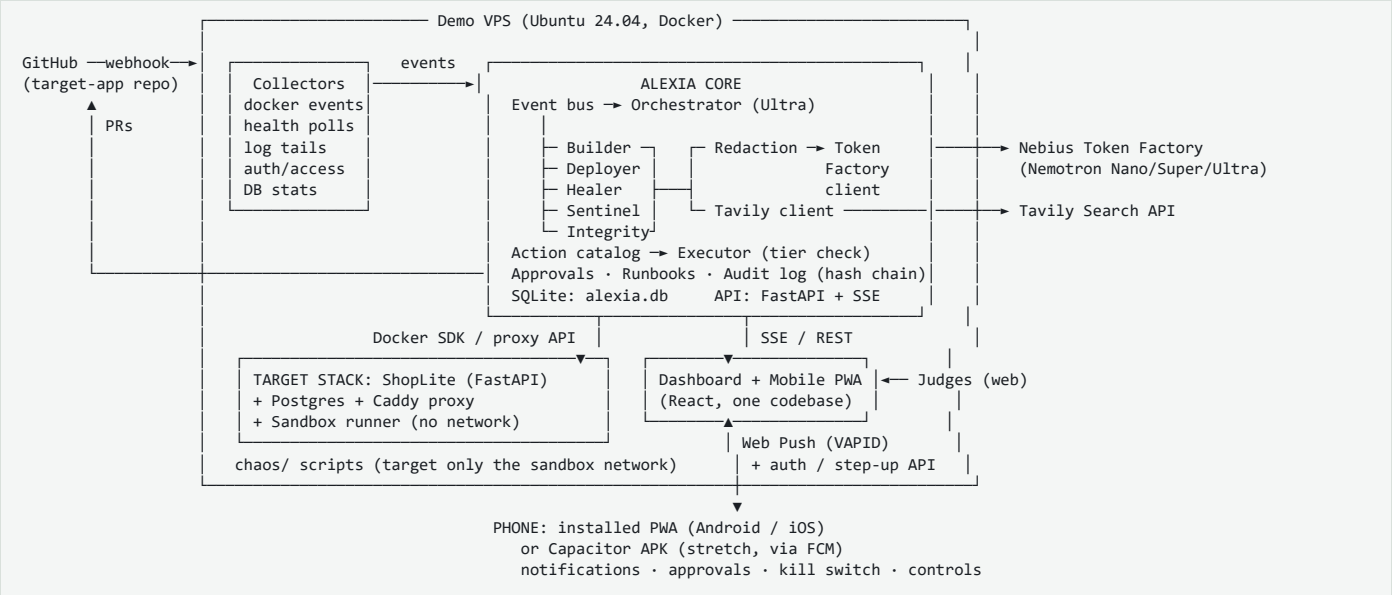
7.3 Nebius services

Service	Use	Pri
Token Factory	All inference (required)	M
Nebius AI Cloud compute VM	Host the demo VPS (if Builder Program credits allow; otherwise any VPS)	S
Nebius Serverless Jobs	Run Builder's sandboxed test suites	C

---

8. Architecture

8.1 System diagram



8.2 Tech stack

Layer	Choice	License
Language	Python 3.11	PSF
API / backend	FastAPI, Uvicorn, Pydantic v2, sse-starlette	MIT / BSD
LLM client	`openai` SDK pointed at the Token Factory base URL	Apache-2.0
Search	`tavily-python`	MIT
Docker control	`docker` SDK for Python	Apache-2.0
GitHub	REST API via `httpx`	BSD
Storage	SQLite (Alexia state), Postgres 16 (target app)	Public domain / PostgreSQL
Dashboard + mobile web app	React + Vite + Tailwind CSS + `vite-plugin-pwa` (manifest + service worker)	MIT
Push notifications	Web Push with VAPID keys via `pywebpush`	MPL-2.0 (used unmodified as a library)
Mobile auth	WebAuthn passkeys via `py_webauthn` (fallback TOTP via `pyotp`); session JWT	BSD / MIT
Native app (stretch)	Capacitor (Android) + Firebase Cloud Messaging	MIT
Proxy / HTTPS	Caddy	Apache-2.0
Target app	ShopLite: small FastAPI shop with users, orders and login	ours (MIT)

8.3 Repository layout (single public monorepo)

```
alexia/
├── LICENSE                    # MIT — first commit
├── README.md
├── THIRD_PARTY.md
├── docker-compose.yml         # alexia + dashboard + target stack
├── .env.example               # never commit .env
├── config/models.yaml         # model IDs + routing thresholds
├── alexia/
│   ├── core/    bus.py schemas.py orchestrator.py llm.py router.py redact.py memory.py audit.py budget.py
│   ├── agents/  builder.py deployer.py healer.py sentinel.py integrity.py
│   ├── collectors/ docker_events.py health.py logs.py authlog.py dbstats.py git_webhook.py
│   ├── actions/ catalog.py executor.py approvals.py
│   ├── tools/   tavily.py github.py sandbox.py snapshots.py reports.py
│   ├── remote/  auth.py (passkey/TOTP, step-up) devices.py push.py controls.py
│   └── api/     main.py sse.py demo.py
├── dashboard/                 # React + Vite — also the installable mobile PWA (src/mobile/ screens, sw.ts)
├── mobile-android/            # (stretch) Capacitor wrapper → APK in GitHub Releases
├── target-app/                # ShopLite + tests + migrations (vulnerable code behind DEMO_VULN=1)
├── chaos/                     # s1_bad_deploy.sh … s7_bad_migration.sh, reset.sh
├── evals/                     # scenario runner: runs all scenarios 3x, outputs a pass/fail table
└── docs/  PRD.md architecture.md feedback.md
```

8.4 Shared contracts (frozen by 23:00 IST on 28 Oct; changes need all three to agree)

```
// Event
{ "id": "evt_01H…", "ts": "2026-10-29T10:15:02Z",
  "source": "collector.docker|collector.health|collector.logs|collector.auth|collector.db|webhook.git",
  "kind": "container_exit|oom|health_fail|error_spike|disk_high|secret_in_diff|bruteforce|sqli_pattern|row_read_anomaly|deploy_request|migration_pending",
  "service": "shoplite-api", "severity": "low|medium|high|critical",
  "data": { } }

// Incident
{ "id": "inc_…", "status": "open|triaged|acting|verifying|resolved|escalated",
  "category": "crash|oom|disk_full|config_error|dependency|code_bug|security|data_integrity|unknown",
  "events": ["evt_…"], "owner_agent": "healer|sentinel|builder|deployer|integrity",
  "diagnosis": { "root_cause": "", "evidence": [], "sources": [{"title":"","url":""}], "confidence": 0.0 },
  "plan": [ /* Action */ ], "snapshots": ["snap_…"], "cost_usd": 0.0, "model_calls": ["call_…"] }

// Action
{ "action": "restart_container", "target": "shoplite-api", "params": {},
  "tier": 0, "reason": "", "expected_outcome": "", "rollback": "restart previous image",
  "requested_by": "agent:healer|user:<id>@device:<id>" }

// Notification (push payload — redacted, no secrets or personal data)
{ "id": "ntf_…", "incident_id": "inc_…", "severity": "critical|high|medium|low|info",
  "title": "SQL injection blocked on shoplite-api", "body": "1 IP blocked · fix PR ready · approval needed",
  "deep_link": "/m/approvals/act_…", "actions": ["review", "reject"] }
```

---

9. Safety, security and data-protection design

9.1 Action catalog and tiers (enforced in `executor.py`, not by prompt)

Tier	Actions	Behaviour
0 — Auto	`restart_container`, `rollback_to_last_good_image`, `clear_tmp_and_rotate_logs`, `block_ip_temporarily` (≤ 1 h), `take_snapshot`, `run_integrity_check`, `open_pull_request`	Executed immediately, with a snapshot first if data could be touched
1 — Approval	`restore_snapshot`, `rotate_secret`, `isolate_container_network`, `apply_config_change`, `raise_memory_limit`, `run_migration`, `merge_pull_request_and_deploy`, `kill_process`	Queued; a human approves or rejects in the dashboard (or Telegram)
2 — Forbidden	Anything not in the catalog: arbitrary shell, deleting volumes, backups or databases, `DROP`/`TRUNCATE` outside the migration guard, opening firewall ports, disabling logging, force-pushing	Rejected and logged as a policy violation

9.2 Protecting the data Alexia handles

- **Data minimisation:** only the relevant log window (± 2 min) is sent to models, after redaction (FR-C4).
- **No real personal data in the demo:** ShopLite is seeded with synthetic users (Faker). Never point the demo at real user data.
- **Secrets:** `.env` only; `.env.example` committed; a `gitleaks` pre-commit hook on our own repo, since Alexia's own repo must not leak a key.
- **Evidence integrity:** evidence files are hashed (sha256) and referenced from the hash-chained audit log.

- **Least privilege:** the GitHub token is fine-grained to the target repo only. The Docker socket is used only by the executor. The sandbox has no network.

9.3 Legal guardrails for attack simulations

- Attacks are simulated **only against containers we own** on the demo VPS. Chaos scripts refuse any target outside a hard-coded allowlist (`shoplite-api`, `localhost`, Docker network `alexia_sandbox`).
- No scanning, brute-forcing or probing of third-party systems. Unauthorised access is an offence under India's IT Act 2000 (Secs. 43 and 66) and equivalent laws elsewhere.
- Vulnerable code in ShopLite exists only when `DEMO_VULN=1`, and the README warns against deploying it publicly with real data.

9.4 Regulatory mapping for breach reports (drafts)

Regime	Deadline	What Alexia pre-fills
CERT-In Directions (Apr 2022)	Report listed incidents within **6 hours** of noticing; keep ICT logs for **180 days** within India	Incident type, time detected, systems affected, indicators (IPs, payloads), actions taken, contact placeholder; log-retention reminder
DPDP Rules 2025 (Rule 7)	Board: initial intimation **without delay**; detailed report **within 72 hours** (extendable on request). Data Principals: without delay	Nature, extent, timing, location and likely impact; cause; mitigation; findings on who caused it; remedial measures; draft notice to affected users in plain language
GDPR Art. 33 / 34	Supervisory authority within **72 hours**; data subjects without undue delay if high risk	Categories and approximate number of data subjects and records, likely consequences, measures taken

All drafts carry: "*DRAFT generated by Alexia. Not legal advice. Review with your legal or compliance contact before submission.*"

9.5 Remote access security (mobile companion)

A phone that can control the server is itself an attack surface, so these rules are enforced server-side:

- **Same catalog and tiers.** The mobile API exposes only catalog actions. Tier 2 stays impossible from the phone, and there is no shell endpoint.
- **Step-up for anything risky.** Approving Tier-1 actions and resuming autonomy need a passkey or biometric (WebAuthn) check in the last 5 minutes; the fallback is TOTP.
- **Single-use, expiring approvals.** Approval tokens are HMAC-signed, bound to the action and device, and expire after 5 minutes, so a replayed request does nothing.
- **Fail-safe direction.** Pausing autonomy is easy; resuming is hard. If the phone is lost, revoke the device from the dashboard; sessions expire after 12 hours.
- **No sensitive data in pushes.** Push payloads go through redaction (FR-C4) and carry only a short summary and a link. Details load inside the authenticated app.
- **Everything is audited.** Every remote action goes into the hash chain with user, device, IP and time, and is visible on the Devices & Security screen.
- **Transport.** HTTPS only (Caddy), HSTS, secure HTTP-only cookies, CSRF protection, rate limits on login and approval endpoints.

---

10. Demo scenarios and acceptance criteria

All scenarios are run by `evals/run_all.py` three times in a row. A scenario passes only if all of its acceptance criteria pass every time.

#	Scenario	Pri	Agents	Acceptance criteria
S1	**Bad deploy** — commit a bug (division by zero in the order total) and push	M	Builder, Deployer, Healer	Builder review flags the risk; if forced through, the health gate fails → rollback < 60 s → Builder PR has a test that fails before and passes after the patch
S2	**Crash** — `docker kill shoplite-api`	M	Healer	Detected < 15 s, restarted, health verified, incident resolved < 60 s, runbook entry created
S3	**Disk full** — fill the log volume to 95 %	S	Healer, Integrity	Snapshot taken, logs rotated and cleaned, disk < 70 %, no data files touched

#	Scenario	Pri	Agents	Acceptance criteria
S4	**Leaked secret** — commit `STRIPE_SECRET_KEY=sk_live_…` (fake value)	M	Sentinel, Builder, Deployer	Deploy blocked; Tier-1 approval to rotate (simulated); Builder PR moves the value to env vars and adds `.gitignore`; history-scrub instructions in the PR
S5	**Brute force** — 500 failed logins against ShopLite `/login` from one container	S	Sentinel	IP blocked < 30 s; incident with timeline; block expires after 1 h
S6	**SQL injection and data exposure** — `/search?q=' UNION SELECT email,phone FROM users--`	M	Sentinel, Integrity, Builder, Deployer, Orchestrator	Detected < 30 s; evidence preserved; IP blocked; exposure assessment (tables, columns, row count); Builder PR switches to a parameterised query plus a payload regression test (red→green); approved merge deploys; DPDP, CERT-In and GDPR drafts generated
S7	**Destructive migration** — migration drops `users.phone`	C	Integrity, Deployer	Dry run detects data loss; deploy blocked pending approval; reviewer sees the row count at risk
S8	**Remote approval from a phone** — during S6, the fix PR needs merge approval	S	Mobile, Deployer	Push arrives on the phone < 10 s after approval is requested; tap → Approvals → biometric/passkey → approved → deploy starts; audit log shows the device ID
S9	**Kill switch** — tap "Pause all autonomy" on the phone, then run S2	S	Mobile, Healer	Healer detects the crash but **queues** the restart for approval instead of acting; resume needs step-up

MVP cut line: S1, S2, S4 and S6 must work by 30 Oct 12:00 IST. Then S8 (phone approval) and S9 (kill switch), which are strong on video. S3 and S5 if time permits; S7 and the native APK are stretch goals.

---

11. Non-functional requirements

ID	Requirement
NFR1	Detection latency < 30 s for rule-based detectors
NFR2	Dashboard updates within 2 s of a state change (SSE)
NFR3	Alexia survives its own restart: state in SQLite; idempotent action execution
NFR4	Public demo stays up until 16 Dec 2026: restart policies, uptime monitor, nightly environment reset
NFR5	One scenario at a time in demo mode, with a global 2-minute cooldown and a daily token budget cap
NFR6	No secrets in the repo, logs or video; keys rotated after the hackathon
NFR7	One-command local setup: `cp .env.example .env && docker compose up`
NFR8	Every model call logged with model, tokens, latency, cost and the redacted prompt hash
NFR9	Push notification delivered < 10 s after the event (Web Push, Android Chrome and installed iOS PWA)
NFR10	Mobile screens usable one-handed at 360 px width; Lighthouse PWA "installable" check passes

---

12. Team split — three people

12.1 Roles

P1 — AI & Agent Lead ("the Brain") · Devpost Representative

Owns everything that talks to a model or writes code.

- Token Factory client, model router, escalation and cost accounting (FR-C3, FR-C5, FR-C10)
- Redaction layer (FR-C4), because it sits at the model boundary
- Shared schemas and contracts (§8.4) — **first deliverable**
- Orchestrator and prompts for every agent (triage, diagnosis, plan, investigation, reports)
- **Builder agent** end to end (FR-B1 to FR-B6), including the sandbox run loop and GitHub PRs
- Tavily integration and caching (§7.2); runbook memory (FR-C9)

- `evals/run_all.py` scenario harness
- Mobile: Nano prompt for one-line, redacted push summaries (FR-M3); notification rules (§6.8 matrix)
- Submission: Devpost description, "how we used Nemotron" README section, feedback section, final submit

P2 — Platform & Reliability Lead ("the Hands")

Owns everything that touches the server, containers and data.

- Demo VPS: Ubuntu, Docker, Caddy (HTTPS), domain, firewall, uptime monitor; keeps it running until 16 Dec
- **ShopLite** target app, Postgres, seed data, tests, `DEMO_VULN` flag
- Collectors (FR-H1) and event publishing
- **Action catalog + executor + approvals enforcement** (FR-C6, FR-C7) and the **audit hash chain** (FR-C8)
- **Deployer agent** (FR-D1 to FR-D5)
- **Healer** actions and verification (FR-H5)
- **Integrity agent** (FR-I1 to FR-I5): snapshots, restore, checks, migration guard, exposure assessment
- `chaos/` scripts plus `reset.sh`, with the allowlist guard
- **Mobile backend** (`alexia/remote/`): login, passkey/TOTP step-up, device registry and revocation, signed single-use approval tokens, kill switch state, Web Push sender with VAPID keys (FR-M2, M5 to M9)
- README: Quickstart, Configuration, Deployment; `THIRD_PARTY.md`

P3 — Security & Product Lead ("the Shield and the Face")

Owns breach detection and everything the judges see.

- **Sentinel detectors** (FR-S1 to FR-S4) and evidence preservation (FR-S6)
- Containment specs (FR-S5), implemented as catalog actions together with P2
- Breach report templates (FR-S8), with P1 writing the model prompts
- **Dashboard** (FR-U1 to FR-U9): timeline, incident detail and reasoning trace, approvals, cost panel, scenario launcher, audit view
- **Mobile companion UI** (FR-M1, M4, M12): the PWA manifest, service worker, push subscription and notification click handling; the six mobile screens; install prompt; demo-mode banner. Stretch: Capacitor Android APK (FR-M11)
- **Demo video:** script, recording, editing, upload to YouTube (public), captions
- Product polish: naming, copy, screenshots for Devpost; project thumbnail

12.2 Interfaces between people (who waits for what)

Producer	Artifact	Consumer	Due (IST)
P1	`schemas.py` (Event, Incident, Action, ModelCall) + `llm.py` stub that returns fake responses	P2, P3	28 Oct 23:00
P2	ShopLite running in Compose + `docker_events` collector emitting events	P1, P3	28 Oct 23:59
P2	`executor.execute(action)` with tier checks	P1, P3	29 Oct 13:00
P1	FastAPI endpoints: `/incidents`, `/incidents/{id}`, `/approvals`, `/stream` (SSE), `/scenarios/{id}/run`	P3	29 Oct 13:00
P3	Sentinel events (`secret_in_diff`, `sqli_pattern`, `bruteforce`) on the bus	P1	29 Oct 19:00
P1	Builder `fix(incident) → PR`	P2 (Deployer)	29 Oct 19:00
P2	Integrity `assess_exposure(query_log)`	P3, P1	29 Oct 22:00
P2	Mobile API: `/auth/*`, `/devices`, `/push/subscribe`, `/approvals/{id}/decide` (step-up), `/autonomy` (pause/resume), `/controls/{action}` + `push.send(Notification)`	P3, P1	29 Oct 22:00
P1	`summarize_for_push(incident) → Notification` (redacted, Nano)	P2	29 Oct 22:00

Rule: until a real component lands, the consumer works against a stub or fake that matches the frozen contract. Nobody waits idle.

12.3 RACI for key deliverables

Deliverable	P1	P2	P3
Contracts and schemas	**A/R**	C	C
Builder agent	**A/R**	C	I
Deployer, Healer, Integrity	C	**A/R**	I
Sentinel + breach reports	C (prompts)	C (actions)	**A/R**
Dashboard	C (API)	I	**A/R**
Mobile companion: UI and PWA	I	C	**A/R**
Mobile companion: auth, push, remote controls	C (summaries)	**A/R**	C
Demo VPS and uptime to 16 Dec	I	**A/R**	I
README	**A** (Nemotron section)	**R** (setup)	R (screenshots)
Video	C	C	**A/R**
Devpost submission and feedback	**A/R**	C	C

A = accountable, R = responsible, C = consulted, I = informed.

---

13. Setup steps — do these first (28 Oct, first 2 hours)

1. **Devpost:** P1 (Representative) creates the project. P2 and P3 accept the team invites. Each member confirms eligibility. Agree the prize split in team chat.

2. **Credits:** each member joins the **Nebius Builder Program** from the hackathon page to get Token Factory and Tavily credits.
3. **Token Factory:** create an API key, then verify the Nemotron model IDs:

```
curl -s https://api.tokenfactory.nebius.com/v1/models -H "Authorization: Bearer $NEBIUS_API_KEY" | grep -io '"id": *"nvidia[^"]*"'
```

4. **Smoke test (P1)**, which must work before anything else is built:

```
import os
from openai import OpenAI

client = OpenAI(base_url="https://api.tokenfactory.nebius.com/v1/", api_key=os.environ["NEBIUS_API_KEY"])
r = client.chat.completions.create(
    model="nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B",   # replace with the ID verified in step 3
    messages=[{"role": "user", "content": "Classify: 'container exited with code 137'. Reply as JSON {category, confidence}."}],
    temperature=0,
)
print(r.choices[0].message.content, r.usage)
```

5. **Also verify on day 1** whether the chosen models support `tools` / function calling and `response_format` on Token Factory. If they don't, use JSON-in-prompt plus Pydantic validation (FR-C5), which is the default plan anyway.
6. **Tavily:** create an API key (free plan: 1,000 credits per month; basic search = 1 credit).
7. **GitHub:** create a public repo named `alexia` with an **MIT LICENSE in the first commit**. Check that "MIT license" appears in the About sidebar. Add a description and topics (`nvidia-nemotron`, `nebius`, `ai-agents`, `devsecops`). Create a second repo, `shoplite`, for the target app if PRs should land there (otherwise use the `target-app/` folder with a path filter).
8. **Mobile (P2 + P3):** generate VAPID keys (`npx web-push generate-vapid-keys` or `pywebpush`); store them in `.env`. Make sure the demo domain has HTTPS (required for service workers, push and passkeys). Have one Android phone and one iPhone (iOS 16.4+) ready for testing.
9. **Fine-grained GitHub token**, scoped to the target repo only (contents read/write, pull requests read/write), and a webhook secret.
10. **VPS (P2):** 4 vCPU / 8 GB / 80 GB, Ubuntu 24.04, Docker + Compose. Open only ports 22 (key auth only), 80 and 443. Caddy with a domain or subdomain for HTTPS. Use Nebius AI Cloud compute if credits allow.
11. **Repo hygiene:** `.gitignore` covers `.env`; add a `gitleaks` pre-commit hook; branch protection on `main` (PRs only).
12. **Feedback log:** create `docs/feedback.md` now. Everyone adds friction points and good surprises as they happen (with the time and model ID).

---

14. Timeline (IST) — 28 Oct → 30 Oct

When	P1 — Brain	P2 — Hands	P3 — Shield and Face	Checkpoint
Wed 28 Oct, 19:00–23:59	Setup steps 1–7; schemas; LLM client + smoke test; model IDs verified	VPS + Docker + Caddy; ShopLite + Postgres + seed; docker-events collector	Dashboard skeleton (React, SSE mock); Sentinel secret-scan rules; video storyboard draft	**CP0 23:59:** Nemotron call works; ShopLite live; contracts frozen
Thu 29 Oct, 09:00–13:00	Router + escalation; Healer triage/diagnose/plan prompts; Tavily client	Executor + tiers + audit chain; health/log collectors; Deployer webhook + build	Timeline + incident detail UI; secret scan on the diff hooked into the Deployer gate	**CP1 13:00:** S2 (crash) end to end, visible in the dashboard
Thu 29 Oct, 14:00–19:00	**Builder**: patch + test + sandbox loop + PR	Health gate + rollback; snapshots + restore; chaos s1/s2/s4	SQLi + brute-force detectors; approvals UI; cost panel	**CP2 19:00:** S1 and S4 end to end
Thu 29 Oct, 20:00–24:00	Orchestrator for S6; investigation + report prompts; runbook memory; push summary prompt	Exposure assessment; chaos s6 + `reset.sh`; **mobile API: auth, step-up, push sender, kill switch**; deploy the full stack to the VPS	Evidence preservation; breach report viewer; scenario launcher; **PWA manifest + service worker + push subscription**	**CP3 24:00:** S6 end to end on the VPS; a test push reaches a phone
Fri 30 Oct, 08:00–12:00	`evals/run_all.py`, 3× runs; fix flakiness; cost guard + replay mode	Device revocation; uptime monitor; restart policies; nightly reset; S3/S5 if possible	**Mobile screens** (Home, Approvals, Controls); S8 + S9 on a real phone; record the video "A-roll" (desktop **and phone screen recording**)	**CP4 12:00: FEATURE FREEZE** (S8/S9 included only if green)
Fri 30 Oct, 12:00–15:00	Devpost description; Nemotron README section; feedback section	README Quickstart and Config; `THIRD_PARTY.md`; fresh-clone test on a clean machine	Edit the video to ≤ 2:50; captions; thumbnail; upload to YouTube as **Public**	**CP5 15:00:** all assets ready
Fri 30 Oct, 15:00–18:00	Fill the Devpost form; tag `v1.0-submission`; **SUBMIT**	Test the demo URL in incognito on mobile and desktop	Proofread everything; check the video plays logged out	**CP6 18:00: SUBMITTED**
18:00–22:30	Buffer only: fix the submission if anything is wrong			**Hard deadline 22:30 IST**
31 Oct → 16 Dec	Answer judge questions	Keep the demo up; monitor the budget	—	Judging 1–15 Dec

Rules of the road

- Hold a 10-minute stand-up at every checkpoint. If a Must item is red at CP2, cut S5 and S7 immediately.
- No new features after CP4; only fixes.
- Every merge to `main` goes through a PR (it's also good demo material).

---

15. Submission package

15.1 Final checklist (P1 runs this before clicking Submit)

- [ ] Track selected: **Coding and Agentic Engineering**
- [ ] Demo URL works logged-out, in incognito, on mobile
- [ ] Mobile companion installs from the demo URL (Android Chrome + iOS Safari "Add to Home Screen"); demo login credentials and a "how to get a push" step are in the testing instructions
- [ ] (If built) Android APK attached to a GitHub Release and linked in the README
- [ ] Public GitHub repo; **MIT license visible in About**; README complete
- [ ] README explains Nemotron (Nano/Super/Ultra) + Token Factory usage, with real token and cost numbers
- [ ] Tavily is called at runtime (visible in the dashboard and the code)
- [ ] YouTube video **Public**, **< 3:00**, no copyrighted music, shows the working project
- [ ] Description covers: what, why, how it works, challenges, what's next

- [ ] Feedback section filled (Token Factory, AI Cloud, Nemotron models)
- [ ] "Built during the submission period" statement (first commit 28 Oct 2026)
- [ ] Testing instructions: URL, "click Run scenario S6", expected result, credentials if any
- [ ] All teammates listed on the Devpost project
- [ ] Commit tagged `v1.0-submission`; demo running from that tag
- [ ] No secrets in the repo (`gitleaks detect` clean)

15.2 Devpost description (draft)

> **Alexia — the DevSecOps guardian that writes, deploys, heals and protects your server.**
>
> **Inspiration.** Small teams run real users' data on a single VPS with no on-call engineer and no security team. When a deploy breaks or an attacker finds an SQL injection, nobody notices for hours, and a rushed fix can destroy data. Indian law (CERT-In: 6 hours; DPDP Rules 2025: 72 hours) and GDPR (72 hours) also give tight deadlines to report breaches.
>
> **What it does.** Five cooperating agents on NVIDIA Nemotron via Nebius Token Factory: - **Builder** writes patches *and* regression tests, runs them in a sandbox until they pass, and opens a PR. - **Deployer** ships code with health gates and automatic rollback. - **Healer** fixes outages and verifies the fix. - **Sentinel** detects breaches, contains them and drafts DPDP / CERT-In / GDPR reports. - **Integrity** snapshots data before every action and guards migrations.
>
> **Mobile companion.** An installable web app (PWA) sends push alerts, lets you approve risky fixes with a fingerprint or passkey, and has a one-tap kill switch that pauses all autonomy.
>
> **How we built it.** Nemotron 3 **Nano** triages every event cheaply, **Super** diagnoses and writes code, and **Ultra** plans, investigates breaches and reviews patches. Tavily grounds each diagnosis in live sources. The model can only choose from a typed, tiered action catalog: no raw shell, risky actions need approval, and forbidden actions are rejected in code. Every step is written to a hash-chained audit log.
>
> **Challenges.** Reliable structured output from the models; making autonomy safe; keeping cost per incident under $0.05.
>
> **What's next.** Kubernetes support, more detectors, SOC 2 evidence export.

15.3 Video script (target 2:50)

Time	Shot	Voice-over (summary)
0:00–0:15	Dashboard calm, then a red alert	Problem: small teams, no on-call, real user data
0:15–0:40	S2 crash → auto-restart (sped up)	"Meet Alexia: five agents on Nemotron via Token Factory"
0:40–1:40	**S6 SQL injection**: detect → IP block → exposure assessment → Builder writes the test (red) → patch → test (green) → PR → **phone buzzes, approve with fingerprint (S8, split screen)** → deploy	Core loop; point out which Nemotron model did each step and the Tavily sources
1:40–2:05	S4 leaked key blocked before deploy; S1 rollback; S9 kill switch on the phone (montage)	Pre-deploy gates, rollback, human stays in control
2:05–2:25	Breach report draft (DPDP/CERT-In), audit chain "verified OK"	Compliance and data integrity
2:25–2:45	Cost panel: Nano / Super / Ultra calls, $ per incident	Smart routing keeps it cheap
2:45–2:50	Logo + repo URL	Call to action

Record each scenario separately, then edit. Use no copyrighted music, and keep API keys, emails and tokens out of every shot.

15.4 README outline

1. What Alexia is (GIF of S6)
2. Quickstart: `cp .env.example .env`, add keys, `docker compose up`, open `http://localhost:8080`
3. Running the demo scenarios (`chaos/` and the dashboard buttons)
4. Architecture (diagram from §8.1)
5. **How Alexia uses NVIDIA Nemotron on Nebius Token Factory** (routing table, escalation, token and cost stats from a real run)
6. Tavily usage
7. Mobile companion: install the PWA, enable notifications, demo login; APK link if built
8. Safety model (tiers, redaction, audit chain, remote-access security)
9. Configuration reference
10. Feedback on Nebius Token Factory, AI Cloud and NVIDIA models

11. Built during the hackathon (dates); license (MIT); third-party licenses

15.5 Feedback section template (aim for "Most Valuable Feedback")

For each item: **what we tried → what happened (model ID, time, error) → why it matters → specific suggestion.** Cover:

- **Token Factory:** onboarding, key creation, model catalog clarity (IDs, context sizes), latency per model, structured-output and tool-calling support, rate limits, error messages, pricing visibility.
- **Nemotron Nano / Super / Ultra:** JSON reliability, code-writing quality, long-context log reading, reasoning quality versus cost.
- **Nebius AI Cloud / Serverless** (if used): setup time, cold starts, docs gaps.

---

16. Risks and mitigations

Risk	Likelihood	Impact	Mitigation	Owner
Judges see it as an "ops tool", not a coding agent (Stage One fail)	Med	**Critical**	Builder writes, runs and tests code front and centre in the video and README; the track wording is quoted in the description	P1
Scope too big for 2.5 days	High	High	MVP cut line (S1, S2, S4, S6); stubs against frozen contracts; checkpoint stand-ups; cut S5 and S7 early	All
Model IDs or features differ from research	Med	Med	Verify on day 1; IDs in config; JSON-in-prompt fallback	P1
Flaky structured output	Med	Med	Pydantic validation, retry, escalate to Super	P1
Agent takes a harmful action	Low	High	Typed catalog, tier enforcement in code, snapshot first, approvals	P2
Public demo exhausts credits or is abused	Med	High	Rate limit, global cooldown, daily budget, replay mode, auto-reset	P2
Demo down during judging (to 15 Dec)	Med	High	Restart policies, uptime monitor alerts to the team, nightly reset; keep the VPS paid	P2
Our own key leaks in the public repo or video	Low	High	gitleaks hook, `.env.example`, blur check before upload, rotate keys after	P3
Video over 3 minutes or unclear	Med	Med	Script timed to 2:50; record per scenario; peer review at CP5	P3
Deadline time-zone confusion	Low	Critical	Everyone uses **22:30 IST**; target submission 18:00 IST	P1
Mobile remote control becomes an attack path	Med	High	Same catalog, step-up auth, single-use signed approvals, device revocation, audit (§9.5)	P2
Push doesn't work on iOS or judges' devices	Med	Low	iOS needs 16.4+ and the PWA installed to the home screen; the in-app notification feed always works; documented in the testing instructions	P3
Mobile work eats core build time	Med	High	Mobile is **Should**: built after CP2; the native APK is stretch only; the PWA reuses dashboard components	P3

---

17. Open decisions

#	Decision	Default if not decided by CP0
D1	Hosting: Nebius AI Cloud VM or another VPS	Nebius VM if credits are available within 1 h, otherwise any VPS
D2	Target app PRs: separate `shoplite` repo or a folder	Separate repo (clearer PR demo)
D3	Mobile: PWA only, or also a native APK	PWA first (website and app in one); Capacitor APK only if CP4 is green early
D3b	Telegram as a notification fallback	Skip unless Web Push fails on our test phones
D4	Nebius Serverless Job for the sandbox	Only if CP3 is green by 22:00 on 29 Oct
D5	Monitor one real team project as a second target	Only after submission, never during the video

---

18. Research appendix (sources, read 28 Oct 2026)

Hackathon

- Official rules: https://nebiusglobalaihackathon.devpost.com/rules. Covers the submission period (ends 30 Oct 2026, 10:00 am PT), the definition of "runs on Nebius", repo license in About, a YouTube video under 3 minutes, testing access until judging ends, Stage One pass/fail, equal-weight criteria, and prize stacking (one Overall or Track award plus one Bonus).
- Hackathon overview page (pasted by the team): tracks, prizes, judges, the Builder Program for credits.

Nebius Token Factory and Nemotron

- Token Factory base URL `https://api.tokenfactory.nebius.com/v1/`, OpenAI-compatible, and the models list endpoint: https://docs.tokenfactory.nebius.com/api-reference/examples/list-of-models.md and https://docs.agno.com/models/providers/gateways/nebius/overview
- Nemotron models on Token Factory (IDs, context, prices): https://nebius.com/services/token-factory/models/nvidia-nemotron-models-inference
- Nemotron 3 Super launch on Token Factory: https://nebius.com/blog/posts/nemotron3-super-now-available
- Nebius Token Factory cookbook, Nemotron: https://github.com/nebius/token-factory-cookbook/tree/main/models/nemotron

Nebius Serverless

- Serverless overview (endpoints and jobs run containers without managing VMs): https://docs.nebius.com/serverless/overview

Tavily

- Credits and pricing (free 1,000 credits per month; basic = 1 credit, advanced = 2): https://docs.tavily.com/documentation/api-credits.md and https://help.tavily.com/articles/8816424538-pricing

Mobile companion (web standards; verify behaviour on test phones)

- Web Push on iOS/iPadOS works only for web apps added to the Home Screen, iOS 16.4+: https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/
- Web Push protocol (RFC 8030) and VAPID (RFC 8292); Python sender `pywebpush`: https://github.com/web-push-libs/pywebpush
- WebAuthn / passkeys for step-up authentication: https://webauthn.guide/ and `py_webauthn`: https://github.com/duo-labs/py_webauthn
- Capacitor (wrap the web app as a native Android app): https://capacitorjs.com/docs

Regulation (for breach report drafts; verify against official text before real use)

- DPDP Rules 2025, Rule 7 (breach intimation: without delay, detailed report within 72 h): https://dpdpa.com/dpdparules/rule7.html and the KPMG guidance at https://assets.kpmg.com/content/dam/kpmgsites/in/pdf/2025/11/dpdp-rules-2025-guidance-to-dpdp-act-implementation.pdf
- CERT-In Directions 2022 (6-hour reporting, 180-day log retention): https://psalegal.com/new-cert-in-directions-overview-and-implications/ and https://www.linklaters.com/insights/blogs/digilinks/2022/may/india-new-and-onerous-cyber-security-framework-and-breach-reporting-o
- GDPR Articles 33 and 34 (72-hour notification to the supervisory authority).

Facts still to verify on day 1: exact model IDs available to our account; tool calling and `response_format` support per model; Builder Program credit amounts; whether Nebius Serverless Jobs fit within credits.

End of document. Owner: P1. Changes to §8.4 contracts or the §10 MVP cut line need agreement from all three team members.